



Práctica final

Documento técnico, API REST Coquito Amarillo.

Estefania Bolivar Marin

Estefania Gomez Gallego

Maria Jose Cardona Velasquez

Facultad de ingeniería, Fundación Universitaria María Cano

Construcción de software I

Luisa Fernando Sanchez Perez

28 de mayo de 2026



Tabla de contenido.

Introducción.....	2
Análisis del problema.	2
1. Descripción de la empresa:	2
2. Descripción del problema:	2
3. Descripción del proyecto:.....	2
4. Objetivo general:.....	3
5. Objetivos específicos:.....	3
6. Alcance del proyecto:	3
7. Actores:	4
8. Requerimientos funcionales:.....	5
9. Requerimientos no funcionales:.....	5
10. Casos de uso:.....	6
Organización del equipo.....	16
Normas y procedimientos.....	17
1. Norma ISO/IEC 25010:.....	17
2. Norma IEEE 730.....	17
Actividades de aseguramiento de calidad.....	18
Matriz de riesgos.	18
Herramientas técnicas.	20
Métricas y criterios de aceptación.....	20
1. Métricas del proyecto:	20
2. Criterios de aceptación:	21
Gestión de cambios.	21
Revisión y aprobación.....	22



Introducción

La empresa Coquito Amarillo S.A.S. enfrenta problemas en la gestión de pedidos debido a la ausencia de un sistema digital robusto. Esta situación genera retrasos en la atención a los clientes, inconsistencias en la información y complejidad para controlar productos y pedidos de forma eficiente. Debido a esto, el cliente solicita construir una WebApp con API-REST que permita gestionar clientes, productos y pedidos, asegurando calidad y escalabilidad.

Análisis del problema.

1. Descripción de la empresa:

Coquito Amarillo S.A.S. es una empresa colombiana dedicada a la producción, comercialización y distribución de productos artesanales de calidad superior enfocado en satisfacer las necesidades de clientes en diferentes regiones del país. La empresa cuenta con presencia comercial en Medellín, Bogotá y Cali.

Gracias a su crecimiento comercial, Coquito Amarillo S.A.S ha logrado incrementar el volumen de clientes, productos y pedidos gestionados diariamente. Debido a esto, la organización presenta dificultades en el manejo eficiente de la información, por lo cual surge la necesidad de implementar una solución que le permita gestionar de manera óptima la información.

2. Descripción del problema:

Actualmente, la empresa Coquito Amarillo S.A.S no cuenta con una plataforma que le permita gestionar de forma eficiente la información de sus clientes, productos y pedidos. El proceso que llevan actualmente ha ocasionado demoras en su gestión y pérdida de información. Adicionalmente, la empresa requiere una plataforma que le asegure calidad y escalabilidad.

3. Descripción del proyecto:

El presente proyecto consiste en el diseño y desarrollo de una WebApp con API-REST para la empresa colombiana Coquito Amarillo S.A.S, con el propósito de gestionar clientes, productos y pedidos.

El desarrollo se realizará bajo la metodología CDIO (concebir, diseñar, implementar y operar), aplicando principios básicos de aseguramiento de calidad de software y prácticas de código limpio.



4. Objetivo general:

Desarrollar una WebApp con API-REST para la gestión de clientes, productos y pedidos de la empresa Coquito Amarillo S.A.S., asegurando calidad y escalabilidad.

5. Objetivos específicos:

- Analizar los requerimientos funcionales y no funcionales relacionados con la gestión de clientes, productos y pedidos de la empresa Coquito Amarillo S.A.S., mediante la elaboración de un documento técnico con mínimo 4 requerimientos funcionales y 3 no funcionales.
- Realizar una matriz de riesgos de software identificando 10 riesgos potenciales y las estrategias que serán implementadas con el fin de la reducción de estas.
- Diseñar la arquitectura y estructura de la API-REST definiendo el modelo de datos y los endpoints, mediante diagramas UML y documentación técnica.
- Implementar la API-REST exitosamente dentro del repositorio del proyecto, registrando los avances mediante los commits realizados, cada integrante del grupo debe tener al menos un commit y en el repositorio se deben encontrar todos los endpoints definidos en la fase de diseño
- Lograr el funcionamiento exitoso de la API-REST en un entorno de prueba local, verificando que las peticiones realizadas en Postman sean exitosas en su totalidad.

6. Alcance del proyecto:

La WebApp con API-REST incluirá:

- Endpoints CRUD para clientes.
- Endpoints CRUD para productos.
- Endpoints CRUD para pedidos.
- Pruebas de funcionamiento de la API.



- Documentación técnica y funcional del proyecto.
- Control de versiones utilizando GitHub.

El proyecto no abarcará:

- Pasarelas de pago.
- Facturación electrónica.
- Interfaz gráfica.
- Implementación de base de datos de información.

7. Actores:

Código	Actor	Rol	Principales intereses	Nivel de interés
AC01	Gerencia Coquito Amarillo S.A.S.	Cliente.	Optimizar la gestión de clientes, productos y pedidos.	Alto
AC02	Administradores	Usuarios directos del proyecto.	Gestionar la información.	Alto
AC03	Equipo de desarrollo	Desarrolladores del proyecto.	Construcción de la WebApp con API REST	Alto
AC04	Clientes de Coquito Amarillo S.A.S.	Usuarios indirectos del proyecto.	Obtener la mejor atención por parte de la empresa Coquito Amarillo S.A.S.	Medio

**8. Requerimientos funcionales:**

Código	Nombre	Descripción	Prioridad
RF01	Estructura de respuestas y peticiones.	Los modelos de los datos que van a ser recibidos y enviados deben estar bien definidos.	Alta
RF02	CRUD clientes	La API debe permitir crear, leer, actualizar y eliminar los datos de los clientes de Coquito Amarillo S.A.S.	Alta
RF03	CRUD productos	La API debe permitir crear, leer, actualizar y eliminar los datos de los productos de Coquito Amarillo S.A.S.	Alta
RF04	CRUD pedidos	La API debe permitir crear, leer, actualizar y eliminar los datos de los pedidos de Coquito Amarillo S.A.S.	Alta

9. Requerimientos no funcionales:

Código	Nombre	Descripción	Prioridad
RNF01	Entorno de desarrollo	La API debe ser desarrollada en Github codespaces	Alta



RNF02	Códigos de estado HTTP	La API debe responder con códigos estandarizados para que el cliente sepa con claridad que ocurrió con su petición.	Media
RNF03	Formato JSON	Tanto las respuestas como las peticiones deben estar exclusivamente en formato JSON.	Alta

10. Casos de uso:

Tabla de resumen de casos de uso:

Código	Código de Actor	Caso de uso	Descripción	Código de requerimiento asociado
CU01	AC02	Registrar clientes	Registrar un nuevo cliente en el sistema mediante solicitud a la API REST.	RF02
CU02	AC02	Consultar clientes	Visualizar datos de los clientes almacenados en el sistema mediante solicitud a la API REST.	RF02
CU03	AC02	Actualizar clientes	Modificar los datos de los clientes existentes en el sistema mediante solicitud a la API REST.	RF02
CU04	AC02	Eliminar clientes	Eliminar los datos de los clientes existentes en el sistema mediante solicitud a la API REST.	RF02
CU05	AC02	Registrar productos	Registrar nuevo producto en el sistema mediante solicitud a la API REST.	RF03



CU06	AC02	Consultar productos	Visualizar datos de los productos almacenados en el sistema mediante solicitud a la API REST.	RF03
CU07	AC02	Actualizar productos	Modificar los datos de los productos existentes en el sistema mediante solicitud a la API REST.	RF03
CU08	AC02	Eliminar productos	Eliminar los datos de los productos existentes en el sistema mediante solicitud a la API REST.	RF03
CU09	AC02	Registrar pedidos	Registrar un nuevo pedido en el sistema mediante solicitud a la API REST.	RF04
CU10	AC02	Consultar pedidos	Visualizar datos de los pedidos almacenados en el sistema mediante solicitud a la API REST.	RF04
CU11	AC02	Actualizar pedidos	Modificar los datos de los pedidos existentes en el sistema mediante solicitud a la API REST.	RF04
CU12	AC02	Eliminar pedidos	Eliminar los datos de los pedidos existentes en el sistema mediante solicitud a la API REST.	RF04

Especificación detallada de casos de uso:

- **Caso de uso CU01:**

- **Nombre:** Registrar clientes.
- **Código de actores:** AC02
- **Descripción:** Registrar un nuevo cliente en el sistema mediante solicitud a la API REST.
- **Código de requerimiento asociado:** RF02
- **Precondiciones:**
 1. El actor tiene acceso activo a Postman o a la WebApp.



2. La API REST debe estar en ejecución.

▪ **Flujo principal**

1. El actor envía una solicitud POST con los datos del cliente nuevo.
2. La API valida la información enviada.
3. Los datos enviados son almacenados.
4. La API responde con una respuesta de éxito.

▪ **Flujo alterno:**

1. El actor envía una solicitud POST con datos inválidos o con datos faltantes.
2. La API responde con un mensaje de error.

▪ **Postcondiciones:**

1. El cliente queda registrado correctamente.

• **Caso de uso CU02:**

▪ **Nombre:** Consultar clientes.

▪ **Código de actores:** AC02

▪ **Descripción:** Visualizar datos de los clientes almacenados en el sistema mediante solicitud a la API REST.

▪ **Código de requerimiento asociado:** RF02

▪ **Precondiciones:**

1. El actor tiene acceso activo a Postman o a la WebApp
2. La API debe estar en ejecución.
3. Deben existir clientes registrados.

▪ **Flujo principal:**

1. El actor envía una solicitud GET.
2. La API busca los registros.



3. La API responde exitosamente y retorna los datos en formato JSON, en caso de que no haya ningún cliente registrado la API responde con una lista vacía.

- **Postcondiciones:**

1. Los datos de los clientes pueden ser visualizada correctamente.

- **Caso de uso CU03:**

- **Nombre:** Actualizar clientes.

- **Código de actores:** AC02

- **Descripción:** Modificar los datos de los clientes existentes en el sistema mediante solicitud a la API REST.

- **Código de requerimiento asociado:** RF02

- **Precondiciones:**

1. El actor tiene acceso activo a Postman o a la WebApp
2. La API debe estar en ejecución.
3. El cliente que se desea modificar debe existir.

- **Flujo principal:**

1. El actor envía una solicitud PUT con los nuevos datos.
2. La API valida la información.
3. Se actualiza la información del cliente.
4. La API responde exitosamente

- **Flujo alterno:**

1. El actor envía una solicitud PUT para modificar datos de un cliente no existente.
2. No existe el cliente ingresado.
3. La API responde con un mensaje de error.

- **Postcondiciones:**

1. Los datos del cliente son modificados.



- **Caso de uso CU04:**
 - **Nombre:** Eliminar clientes.
 - **Código de actores:** AC03.
 - **Descripción:** Eliminar los datos de los clientes existentes en el sistema mediante solicitud a la API REST.
 - **Código de requerimiento asociado:** RF02
 - **Precondiciones:**
 1. El actor tiene acceso activo a Postman o a la WebApp
 2. La API debe estar en ejecución.
 3. El cliente que se desea eliminar debe existir.
 - **Flujo principal:**
 1. El actor envía una solicitud DELETE.
 2. La API elimina al cliente seleccionado.
 3. La API responde exitosamente
 - **Flujo alternativo:**
 1. El actor envía una solicitud DELETE para eliminar a un cliente no existente.
 2. No existe el cliente ingresado.
 3. La API responde con un mensaje de error.
 - **Postcondiciones:**
 1. El cliente es eliminado.
- **Caso de uso CU05:**
 - **Nombre:** Registrar productos.
 - **Código de actores:** AC03.
 - **Descripción:** Registrar nuevo producto en el sistema mediante solicitud a la API REST.
 - **Código de requerimiento asociado:** RF03
 - **Precondiciones:**



1. El actor tiene acceso activo a Postman o a la WebApp
 2. La API debe estar en ejecución.
- **Flujo principal:**
 1. El actor envía una solicitud POST con los datos del producto nuevo.
 2. La API valida la información enviada.
 3. Los datos enviados son almacenados.
 4. La API responde con una respuesta de éxito.
 - **Flujo alternativo:**
 1. El actor envía una solicitud POST con datos inválidos o con datos faltantes.
 2. La API responde con un mensaje de error.
 - **Postcondiciones:**
 1. El producto es registrado.
- **Caso de uso CU06:**
 - **Nombre:** Consultar productos.
 - **Código de actores:** AC03.
 - **Descripción:** Visualizar datos de los pedidos almacenados en el sistema mediante solicitud a la API REST.
 - **Código de requerimiento asociado:** RF03
 - **Precondiciones:**
 1. El actor tiene acceso activo a Postman o a la WebApp
 2. La API debe estar en ejecución.
 3. Deben existir productos registrados.
 - **Flujo principal:**
 1. El actor envía una solicitud GET.
 2. La API busca los registros.



4. La API responde exitosamente y retorna los datos en formato JSON, en caso de que no haya ningún producto registrado la API responde con una lista vacía.

- **Postcondiciones:**

1. Los datos de los productos pueden ser visualizada correctamente.

- **Caso de uso CU07:**

- **Nombre:** Actualizar productos.

- **Código de actores:** AC02

- **Descripción:** Modificar los datos de los productos existentes en el sistema mediante solicitud a la API REST.

- **Código de requerimiento asociado:** RF03

- **Precondiciones:**

1. El actor tiene acceso activo a Postman o a la WebApp
2. La API debe estar en ejecución.
3. El producto que se desea modificar debe existir.

- **Flujo principal:**

1. El actor envía una solicitud PUT con los nuevos datos.
2. La API valida la información.
3. Se actualiza la información del producto.
4. La API responde exitosamente

- **Flujo alterno:**

1. El actor envía una solicitud PUT para modificar datos de un producto no existente.
2. No existe el producto ingresado.
3. La API responde con un mensaje de error.

- **Postcondiciones:**

1. Los datos del producto son modificados.

- **Caso de uso CU08:**



- **Nombre:** Eliminar productos.
- **Código de actores:** AC03.
- **Descripción:** Eliminar los datos de los productos existentes en el sistema mediante solicitud a la API REST.
- **Código de requerimiento asociado:** RF03
- **Precondiciones:**
 1. El actor tiene acceso activo a Postman o a la WebApp
 2. La API debe estar en ejecución.
 3. El producto que se desea eliminar debe existir.
- **Flujo principal:**
 1. El actor envía una solicitud DELETE.
 2. La API elimina al producto seleccionado.
 3. La API responde exitosamente
- **Flujo alternativo:**
 1. El actor envía una solicitud DELETE para eliminar a un producto no existente.
 2. No existe el producto ingresado.
 3. La API responde con un mensaje de error.
- **Postcondiciones:**
 1. El producto es eliminado.
- **Caso de uso CU09:**
 - **Nombre:** Registrar pedidos.
 - **Código de actores:** AC03.
 - **Descripción:** Registrar nuevo pedido en el sistema mediante solicitud a la API REST.
 - **Código de requerimiento asociado:** RF04
 - **Precondiciones:**
 1. El actor tiene acceso activo a Postman o a la WebApp



2. La API debe estar en ejecución.
3. Debe de existir al menos un cliente registrado.
4. El cliente que realiza el pedido debe de corresponder a un cliente existente.
5. Cada producto de la lista debe corresponder a un producto existente.

- **Flujo principal:**

1. El actor envía una solicitud POST con los datos del pedido nuevo.
2. La API valida la información enviada.
3. Los datos enviados son almacenados.
4. La API responde con una respuesta de éxito.

- **Flujo alterno:**

1. El actor envía una solicitud POST con datos inválidos o con datos faltantes.
2. La API responde con un mensaje de error.

- **Postcondiciones:**

1. El pedido es registrado.

- **Caso de uso CU10:**

- **Nombre:** Consultar pedidos.
- **Código de actores:** AC03.
- **Descripción:** Visualizar datos de los pedidos almacenados en el sistema mediante solicitud a la API REST.
- **Código de requerimiento asociado:** RF04
- **Precondiciones:**
 1. El actor tiene acceso activo a Postman o a la WebApp
 2. La API debe estar en ejecución.
 3. Deben existir pedidos registrados.
- **Flujo principal:**
 1. El actor envía una solicitud GET.



2. La API busca los registros.
 5. La API responde exitosamente y retorna los datos en formato JSON, en caso de que no haya ningún pedido registrado la API responde con una lista vacía.
- **Postcondiciones:**
 1. Los datos de los pedidos pueden ser visualizada correctamente.
 - **Caso de uso CU11:**
 - **Nombre:** Actualizar pedidos.
 - **Código de actores:** AC02
 - **Descripción:** Modificar los datos de los pedidos existentes en el sistema mediante solicitud a la API REST.
 - **Código de requerimiento asociado:** RF04
 - **Precondiciones:**
 1. El actor tiene acceso activo a Postman o a la WebApp
 2. La API debe estar en ejecución.
 3. El pedido que se desea modificar debe existir.
 - **Flujo principal:**
 1. El actor envía una solicitud PUT con los nuevos datos.
 2. La API valida la información.
 3. Se actualiza la información del pedido.
 4. La API responde exitosamente
 - **Flujo alternativo:**
 1. El actor envía una solicitud PUT para modificar datos de un pedido no existente.
 2. No existe el producto ingresado.
 3. La API responde con un mensaje de error.
 - **Postcondiciones:**
 1. Los datos del pedido son modificados.



- **Caso de uso CU12:**
 - **Nombre:** Eliminar pedidos.
 - **Código de actores:** AC03.
 - **Descripción:** Eliminar los datos de los pedidos existentes en el sistema mediante solicitud a la API REST.
 - **Código de requerimiento asociado:** RF04
 - **Precondiciones:**
 1. El actor tiene acceso activo a Postman o a la WebApp
 2. La API debe estar en ejecución.
 3. El pedido que se desea eliminar debe existir.
 - **Flujo principal:**
 1. El actor envía una solicitud DELETE.
 2. La API elimina al pedido seleccionado.
 3. La API responde exitosamente
 - **Flujo alternativo:**
 1. El actor envía una solicitud DELETE para eliminar a un pedido no existente.
 2. No existe el producto ingresado.
 3. La API responde con un mensaje de error.
 - **Postcondiciones:**
 1. El pedido es eliminado.

Organización del equipo.

El proyecto será desarrollado por tres integrantes, quienes van a trabajar de manera colaborativa tanto en la documentación como desarrollo técnico de la API REST. Aunque a cada una se le asignará un rol principal con el fin de que esa persona lleve un control riguroso sobre los respectivos entregables.

Estefania Bolivar Marín: Desarrollo.

Estefania Gomez Gallego: Pruebas.

**Maria Jose Cardona Velasquez: Documentación técnica.**

En la siguiente tabla se mostrarán las diferentes responsabilidades dentro de los entregables y la distribución de cada integrante:

Responsabilidades	Estefania Bolivar	Estefania Gomez	Maria Cardona
Redactar documento técnico en Word	Apoya	Apoya	Líder
Desarrollo de API REST	Líder	Apoya	Apoya
Crear y ejecutar pruebas en Postman	Apoya	Líder	Apoya
Documentar API	Apoya	Apoya	Líder
Manuales del proyecto	Apoya	Apoya	Líder
Presentación oral	Todas	Todas	Todas

Normas y procedimientos.**1. Norma ISO/IEC 25010:**

La norma ISO/IEC 25010 será utilizada como referencia para evaluar la calidad de la API REST desarrollada en el proyecto. Esta norma permite analizar características importantes del software como la fiabilidad, mantenibilidad, eficiencia, compatibilidad y seguridad.

- Validación del correcto funcionamiento de los endpoints.
- Verificación de respuestas en formato JSON.
- Evaluación de la eficiencia de las respuestas de la API.
- Control de errores mediante códigos de estado HTTP.

2. Norma IEEE 730

La norma IEEE 730 será utilizada como guía para las actividades de aseguramiento de calidad del software durante el desarrollo del proyecto.

- Planeación de pruebas de la API REST.
- Verificación del cumplimiento de los requerimientos funcionales.
- Documentación de pruebas realizadas en Postman.



-
- Seguimiento de errores detectados durante el desarrollo.
 - Validación de los resultados obtenidos en cada endpoint implementado.

Actividades de aseguramiento de calidad.

Para garantizar la calidad de la WebApp con API REST desarrollada para la empresa Coquito Amarillo S.A.S., se realizarán las siguientes actividades:

- Verificación del funcionamiento correcto de los endpoints mediante pruebas en Postman.
- Validación de las respuestas y peticiones en formato JSON.
- Revisión de códigos de estado HTTP para comprobar el manejo adecuado de errores y respuestas exitosas.
- Validación de datos enviados a la API para evitar registros inválidos o incompletos.
- Uso de control de versiones mediante GitHub para llevar seguimiento de los cambios realizados en el proyecto.
- Revisión colaborativa del código desarrollado por los integrantes del equipo.
- Actualización continua de la documentación técnica y de los endpoints implementados.
- Ejecución de pruebas funcionales para comprobar las operaciones CRUD de clientes, productos y pedidos.
- Identificación y corrección de errores encontrados durante el desarrollo del proyecto.

Matriz de riesgos.

Riesgo	Probabilidad	Impacto	Nivel	Mitigación
--------	--------------	---------	-------	------------



Caída del servidor o de la API REST	Alta (3)	Alta (3)	9 - Crítico	Usar un entorno estable y realizar monitoreo constante.
Perdida de la información de clientes, productos o pedidos.	Media (2)	Alta (3)	6 - Alto	Realizar copias de seguridad frecuentes.
Problemas de seguridad o acceso no autorizado.	Media (2)	Alta (3)	6 - Alto	Implementar validaciones y control de acceso en la API.
Lentitud en las respuestas de la API.	Media (2)	Media (2)	4 - Medio	Realizar pruebas de rendimiento y optimizar los endpoints.
Errores en la conexión entre la WebApp y la API.	Alta (3)	Media (2)	6 - Alto	Realizar pruebas constantes de integración.
Información incorrecta en productos o pedidos.	Media (2)	Media (2)	4 - Medio	Validar los datos ingresados antes de almacenarlos.
Usuarios presentan dificultades al consumir la API.	Baja (1)	Media (2)	2 - Bajo	Documentar correctamente los endpoints y ejemplos de uso.
Fallos en la validación de datos enviados a la API.	Media (2)	Alta (3)	6 - Alto	Implementar validaciones en las peticiones y realizar pruebas funcionales.
Errores en las respuestas JSON de la API.	Media (2)	Alta (3)	6 - Alto	Validar las respuestas mediante pruebas en Postman.
Fallos al consumir los endpoints de la API.	Media (2)	Media (2)	4 - Medio	Verificar el funcionamiento de las rutas mediante pruebas en Postman.

La matriz de riesgo permite identificar posibles problemas que podrían afectar el desarrollo y funcionamiento de la tienda virtual de artesanías. Con esta herramienta se analizan los riesgos según su probabilidad de ocurrencia y el impacto que podrían generar en el sistema.



Herramientas técnicas.

Las herramientas técnicas que serán utilizadas para el desarrollo de la WebApp con API-REST para la empresa colombiana Coquito Amarillo S.A.S. son las siguientes:

- Python: El lenguaje de programación utilizado para desarrollar el proyecto.
- GitHub Codespaces: Entorno de desarrollo en la nube utilizado no solo para alojar el proyecto, sino también para ejecutar de forma local la API.
- GitHub: Controlar las versiones del proyecto.
- Postman: Consumir los endpoints de la API para poder validar de forma local que esta está funcionando correctamente.
- Microsoft Word: Procesador de texto utilizado para redactar el documento técnico.
- Flask: Framework utilizado para la creación y configuración de la aplicación backend.
- Flask-RESTful: Librería utilizada para facilitar la construcción de endpoints y servicios REST dentro de la API.

Métricas y criterios de aceptación.

1. Métricas del proyecto:

Métrica	Descripción	Indicador
Endpoints	El porcentaje de endpoints desarrollados respecto a los definidos en la etapa de diseño.	100%
Éxito de endpoints	El porcentaje de las rutas que al ejecutar en Postman respondan con códigos de estado HTTP	100%
Requisitos funcionales	Los requerimientos funcionales que están completamente cubiertos.	4
Formato JSON	El porcentaje de respuestas de la API que devuelven la información en formato JSON	100%
Versión del código	Porcentaje de código gestionado mediante el control de versiones.	100%



Documentación técnica.	El porcentaje de ítems documentados respecto a los definidos en el documento entregado por el profesor.	100%
------------------------	---	------

2. Criterios de aceptación:

Criterio de aceptación	Resultado esperado
La API REST permite realizar operaciones CRUD de clientes	Las solicitudes GET, POST, PUT y DELETE funcionan correctamente
La API REST permite realizar operaciones CRUD de productos	Los endpoints responden correctamente en Postman
La API REST permite realizar operaciones CRUD de pedidos	Los pedidos pueden registrarse, consultarse, actualizarse y eliminarse
Las respuestas de la API se entregan en formato JSON	Todas las respuestas retornan información válida en formato JSON
Los endpoints responden con códigos HTTP adecuados	La API utiliza códigos como 200, 201, 400 y 404 correctamente
La API valida datos incorrectos o incompletos	El sistema responde con mensajes de error cuando la información es inválida
El proyecto se encuentra gestionado mediante GitHub	Los avances del proyecto están registrados mediante commits
La documentación técnica está completa	El documento incluye los apartados solicitados por el profesor
Las pruebas realizadas en Postman son exitosas	Los endpoints funcionan correctamente en el entorno local
La API REST se ejecuta correctamente en el entorno de desarrollo	El sistema funciona sin errores críticos en Github Codespaces

Gestión de cambios.

Los cambios realizados durante el desarrollo de la API REST serán controlados mediante GitHub y el uso de control de versiones.

Cada modificación realizada en el proyecto deberá ser validada mediante pruebas en Postman para comprobar el correcto funcionamiento de los endpoints y evitar errores en el sistema.



Adicionalmente, cualquier cambio importante realizado en funcionalidades, endpoints o documentación técnica deberá ser actualizado dentro del proyecto mediante commits con la descripción adecuada del cambio para mantener la consistencia de la información.

Revisión y aprobación.

Actividad	Responsable	Resultado esperado
Revisión de requerimientos funcionales	Estefania Bolivar Marin	Verificar el correcto funcionamiento de los endpoints implementados
Revisión de pruebas funcionales	Estefania Gomez Gallego	Validar las respuestas y funcionamiento de la API REST
Revisión de la documentación técnica	Maria Jose Cardona Velasquez	Confirmar que el documento técnico contenga los apartados solicitados.
Aprobación final del proyecto	Equipo de trabajo	Validar la entrega final del proyecto y su documentación.